

Implementación Técnica del Backend

Sistema de Administración de Finanzas Personales con OCR

FINARA — Módulo: API REST Node.js

Documento de Implementación Técnica
Trabajo Terminal **2026-A062**

Equipo de Desarrollo

Alumno 1: [Nombre completo]

Boleta: [Número de boleta]

Alumno 2: [Nombre completo]

Boleta: [Número de boleta]

Director: [Nombre del director TT]

Versión del documento: 4.0.0

Fecha: Marzo 2026

Estado: Implementación completa

Ciudad de México, México · 2026

Índice general

Lista de Tablas	viii
Lista de Figuras	ix
Lista de Fragmentos de Código	xi
1. Introducción	1
1.1. Propósito del Documento	1
1.2. Alcance	1
1.2.1. Incluido en este Documento	1
1.2.2. Fuera de Alcance	2
1.3. Audiencia Objetivo	2
1.4. Contexto del Proyecto	2
1.4.1. Problemática	2
1.4.2. Solución Propuesta	3
1.4.3. Arquitectura General del Sistema	3
1.5. Versiones del Sistema	3
2. Stack Tecnológico	5
2.1. Runtime y Framework Principal	5
2.1.1. Node.js	5
2.1.2. Express.js	5
2.2. Base de Datos	6
2.2.1. MySQL 8.0	6

2.3. Dependencias de Producción	7
2.4. Dependencias de Desarrollo	7
2.5. package.json Completo	8
2.6. Stack OCR (Python)	9
2.7. Herramientas de Desarrollo	9
3. Arquitectura del Backend	11
3.1. Patrón Arquitectónico	11
3.2. Estructura de Carpetas Completa	11
3.3. Descripción de Módulos Principales	13
3.4. Flujo de una Solicitud HTTP	16
3.5. Formato Estándar de Respuestas	17
3.6. Flujo Especial: Escaneo OCR de Ticket	17
4. Documentación de Endpoints	19
4.1. Autenticación (/api/auth)	20
4.1.1. POST /api/auth/register	20
4.1.2. POST /api/auth/login	22
4.1.3. POST /api/auth/logout	22
4.1.4. GET /api/auth/verify	23
4.1.5. PUT /api/auth/change-password	23
4.2. Usuarios (/api/users)	23
4.2.1. GET /api/users/me	23
4.2.2. PUT /api/users/me	23
4.2.3. DELETE /api/users/me	24
4.2.4. POST /api/users/initial-config	24
4.2.5. GET /api/users/initial-config	24
4.3. Categorías (/api/categories)	24
4.3.1. GET /api/categories	24
4.3.2. GET /api/categories/:id	25

4.3.3.	POST /api/categories	25
4.3.4.	PUT /api/categories/:id	25
4.3.5.	DELETE /api/categories/:id	25
4.4.	Presupuestos (/api/budgets)	25
4.4.1.	GET /api/budgets	25
4.4.2.	GET /api/budgets/category/:categoryId	25
4.4.3.	POST /api/budgets	26
4.4.4.	PUT /api/budgets/:id	26
4.4.5.	DELETE /api/budgets/:id	26
4.5.	Ingresos (/api/incomes)	26
4.5.1.	GET /api/incomes	26
4.5.2.	GET /api/incomes/total	26
4.5.3.	POST /api/incomes	26
4.5.4.	PUT /api/incomes/:id y DELETE /api/incomes/:id	27
4.6.	Gastos (/api/expenses)	27
4.6.1.	GET /api/expenses	27
4.6.2.	GET /api/expenses/recent	27
4.6.3.	GET /api/expenses/category/:categoryId	27
4.6.4.	POST /api/expenses	27
4.6.5.	DELETE /api/expenses/:id	28
4.7.	Tickets OCR (/api/tickets)	28
4.7.1.	POST /api/tickets/scan	28
4.7.2.	GET /api/tickets	29
4.7.3.	GET /api/tickets/:id	29
4.7.4.	GET /api/tickets/:id/products	29
4.7.5.	DELETE /api/tickets/:id	29
4.8.	Deudas (/api/debts)	29
4.8.1.	GET /api/debts	29
4.8.2.	POST /api/debts	29

4.8.3. PUT /api/debts/:id/pay -- Registrar Pago	30
4.9. Alertas (/api/alerts)	30
4.9.1. GET /api/alerts/unread	30
4.9.2. PUT /api/alerts/mark-all-read	30
4.9.3. GET /api/alerts/config y PUT /api/alerts/config	30
4.10. Dashboard (/api/dashboard)	31
4.10.1. GET /api/dashboard/summary	31
4.10.2. GET /api/dashboard/charts/expenses	31
4.10.3. GET /api/dashboard/charts/budgets	31
5. Base de Datos	32
5.1. Visión General	32
5.2. Descripción de Tablas	33
5.2.1. Tabla: Usuarios	33
5.2.2. Tabla: Categorías	33
5.2.3. Tabla: Presupuestos	34
5.2.4. Tabla: Ingresos	34
5.2.5. Tabla: Gastos	35
5.2.6. Tabla: Tickets	35
5.2.7. Tabla: Productos_Tickets	35
5.2.8. Tabla: Deudas	36
5.2.9. Tabla: Alertas	36
5.2.10. Tabla: Configuración_Notificaciones	37
5.2.11. Tabla: Configuración_Inicial	37
5.3. Triggers Automáticos	37
5.3.1. Trigger: actualizar_presupuesto_gasto	37
5.3.2. Trigger: verificar_presupuesto_excedido	38
5.4. Diagrama Entidad-Relación	39
6. Autenticación y Seguridad	40

6.1. Estrategia de Autenticación	40
6.1.1. JSON Web Token (JWT)	40
6.1.2. Hash de Contraseñas con bcrypt	41
6.2. Middleware de Autenticación	41
6.3. Validación de Inputs con express-validator	42
6.4. Sanitización de Inputs	43
6.5. Rate Limiting	43
6.6. Headers de Seguridad	44
6.7. Prevención de Inyección SQL	44
6.8. Resumen de Medidas de Seguridad	45
7. Decisiones de Diseño Técnico	46
7.1. ¿Por qué Node.js?	46
7.2. ¿Por qué Express.js?	47
7.3. ¿Por qué MySQL?	47
7.4. ¿Por qué bcrypt?	48
7.5. ¿Por qué Arquitectura por Capas y no Clean Architecture?	48
7.6. Decisión de OCR: Python sobre Node.js	49
8. Integración con Frontend	50
8.1. Contexto de Integración	50
8.2. Configuración CORS	50
8.3. Consumo desde la App Móvil Flutter	51
8.3.1. Autenticación y Almacenamiento del Token	51
8.3.2. Request Autenticado Estándar	51
8.3.3. Subida de Imagen para OCR	52
8.4. Manejo de Errores desde el Cliente	53
8.5. Formato de Fechas	53
8.6. URL Base de la API	53
9. Historial de Versiones	55

9.1. Versión 4.0.0 (Marzo 2026)	55
9.2. Versión 3.0.0 (Febrero 2026)	56
9.3. Versión 2.0.0 (Febrero 2026)	57
9.4. Versión 1.0.0 (Enero 2026)	57
9.5. Roadmap Futuro	58
Referencias Bibliográficas	59

Índice de tablas

1.1. Audiencias del documento y uso esperado	2
1.2. Versiones del backend FINARA	4
2.1. Middleware Express configurados en app.js	5
2.2. Dependencias de producción del backend FINARA	7
2.3. Dependencias de desarrollo del backend FINARA	7
2.4. Dependencias Python del módulo OCR	9
3.1. Responsabilidad de cada controlador	13
4.1. Resumen de todos los endpoints del backend FINARA	19
5.1. Tablas de la base de datos finara_db	32
5.2. Columnas de la tabla Usuarios	33
6.1. Configuración de rate limiting por contexto	43
6.2. Resumen de medidas de seguridad implementadas	45
7.1. Comparativa de tecnologías backend consideradas	46
7.2. Comparativa de frameworks Node.js	47
7.3. Comparativa de bases de datos consideradas	47
7.4. Comparativa de algoritmos de hash de contraseñas	48
7.5. Justificación de Python para el módulo OCR	49
8.1. Códigos HTTP de la API y su interpretación en Flutter	53
8.2. URL base según entorno	53

9.1. Mejoras planificadas para fases futuras 58

Índice de figuras

1.1. Arquitectura general del sistema FINARA	3
3.1. Flujo de procesamiento de una solicitud HTTP en FINARA Backend .	16
5.1. Diagrama Entidad-Relación simplificado de finara_db	39

Listings

2.1.	Configuración del pool de conexiones MySQL	6
2.2.	package.json del backend FINARA v4.0.0	8
3.1.	Árbol de directorios del backend FINARA	11
3.2.	Formatos de respuesta estándar de la API	17
3.3.	Flujo completo de escaneo de ticket con OCR	17
5.1.	DDL de la tabla Usuarios	33
5.2.	DDL de la tabla Categorías	33
5.3.	DDL de la tabla Presupuestos	34
5.4.	DDL de la tabla Ingresos	34
5.5.	DDL de la tabla Gastos	35
5.6.	DDL de la tabla Tickets	35
5.7.	DDL de la tabla Productos_Tickets	35
5.8.	DDL de la tabla Deudas	36
5.9.	DDL de la tabla Alertas	36
5.10.	DDL de Configuración_Notificaciones	37
5.11.	DDL de Configuración_Inicial	37
5.12.	Trigger de actualización de presupuesto al registrar gasto	38
5.13.	Trigger de alerta cuando el presupuesto alcanza el 90 %	38
6.1.	Estructura del JWT generado por FINARA	40
6.2.	Generación del token JWT en authController.js	40
6.3.	Hash y verificación de contraseña con bcrypt	41
6.4.	Middleware de autenticación JWT completo	41
6.5.	Validaciones del endpoint de registro	42
6.6.	Middleware sanitizador de inputs	43
6.7.	Headers de seguridad HTTP configurados manualmente	44
6.8.	Prepared statement con mysql2	44
8.1.	Configuración CORS en app.js	50
8.2.	Variable de entorno para producción	50
8.3.	Login y almacenamiento del JWT en Flutter (Dart)	51
8.4.	Patrón de request autenticado en Flutter	51

8.5. Envío de imagen de ticket para escaneo OCR 52

Capítulo 1

Introducción

1.1. Propósito del Documento

El presente documento describe la implementación técnica completa del *backend* del sistema FINARA, una aplicación de administración de finanzas personales con capacidad de escaneo y reconocimiento óptico de caracteres (OCR) aplicado a tickets de compra.

Este documento tiene tres propósitos principales:

1. **Referencia de implementación:** Servir como guía técnica detallada para el equipo de desarrollo, describiendo cada módulo, endpoint y decisión de diseño tomada durante la construcción de la API.
2. **Evidencia académica:** Constituir la documentación formal de implementación del módulo backend del Trabajo Terminal **2026-A062** ante la Escuela Superior de Cómputo del Instituto Politécnico Nacional.
3. **Guía de integración:** Proporcionar a los desarrolladores del frontend (app móvil Flutter y futura plataforma web React) la especificación técnica necesaria para consumir correctamente los 46 endpoints de la API REST.

1.2. Alcance

1.2.1. Incluido en este Documento

- API REST completa: 46 endpoints en 10 módulos funcionales.
- Base de datos MySQL: 10 tablas, relaciones, índices y 2 triggers automáticos.

- Sistema de autenticación con JWT y cifrado de contraseñas con bcrypt.
- Pipeline de procesamiento OCR: integración Node.js ↔ Python/Tesseract.
- Middleware de seguridad: rate limiting, sanitización de inputs, headers de protección.
- Suite de pruebas unitarias: 5 módulos de tests con base de datos mockeada.

1.2.2. Fuera de Alcance

- **App móvil Flutter:** documentada en el documento paralelo *“Implementación Técnica — App Móvil Flutter”*.
- **Plataforma web React:** pendiente de implementación (fase futura).
- **Infraestructura de despliegue:** configuración de servidores de producción.

1.3. Audiencia Objetivo

Tabla 1.1: Audiencias del documento y uso esperado

Audiencia	Uso del Documento
Equipo backend	Referencia de implementación, mantenimiento y extensión de módulos.
Equipo frontend	Especificación de endpoints, formatos JSON y códigos de error para integración.
Profesores TT	Evaluación del alcance técnico y calidad de la implementación.
Futuros colaboradores	Incorporación al proyecto con contexto arquitectónico completo.

1.4. Contexto del Proyecto

1.4.1. Problemática

La administración de finanzas personales requiere un registro sistemático y constante de ingresos y gastos. Sin embargo, el proceso manual de captura de datos presenta las

siguientes limitaciones:

- **Fricción de entrada:** registrar cada gasto manualmente consume tiempo y genera abandono del hábito.
- **Pérdida de información:** los tickets físicos se extravían antes de registrar los datos.
- **Desorganización categórica:** sin categorización automática, el análisis de hábitos de gasto es impreciso.

1.4.2. Solución Propuesta

FINARA propone un sistema que automatiza la captura de gastos mediante OCR sobre fotografías de tickets, complementado con un módulo de administración de presupuestos, deudas e ingresos. El backend Node.js actúa como el núcleo del sistema, orquestando la lógica de negocio, la persistencia de datos y la comunicación con el motor OCR Python/Tesseract.

1.4.3. Arquitectura General del Sistema

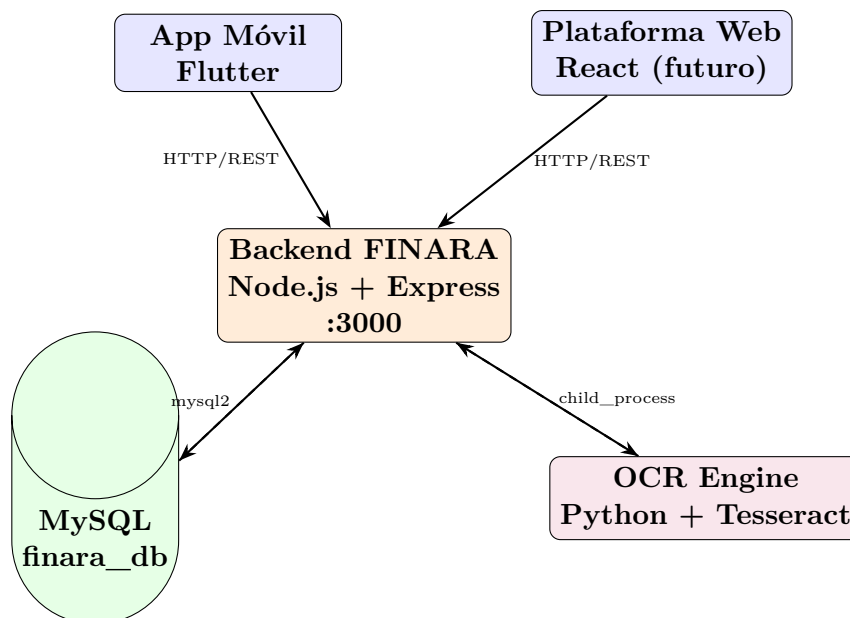


Figura 1.1: Arquitectura general del sistema FINARA

1.5. Versiones del Sistema

Tabla 1.2: Versiones del backend FINARA

Versión	Fecha	Contenido
1.0.0	Enero 2026	Auth, BD, estructura base
2.0.0	Febrero 2026	Módulos financieros (categorías, presupuestos, ingresos, gastos, dashboard)
3.0.0	Febrero 2026	OCR, tickets, deudas, alertas, usuarios
4.0.0	Marzo 2026	Seguridad, testing, optimizaciones, OCR v3

Capítulo 2

Stack Tecnológico

2.1. Runtime y Framework Principal

2.1.1. Node.js

Node.js es un entorno de ejecución de JavaScript del lado del servidor construido sobre el motor V8 de Google Chrome. Para el backend de FINARA se utilizó la versión **18.x LTS** (Long-Term Support), garantizando estabilidad y soporte de seguridad a largo plazo [1].

Nota Técnica

Node.js implementa un modelo de I/O no bloqueante basado en el *event loop*, lo cual lo hace especialmente eficiente para APIs REST donde las operaciones dominantes son lecturas y escrituras a base de datos, no cómputo intensivo [2].

2.1.2. Express.js

Express.js 4.18.x es el framework web minimalista utilizado sobre Node.js [3]. Proporciona el enrutamiento HTTP, el sistema de middleware y el manejo de errores del backend FINARA.

Los middleware de Express utilizados en el proyecto se muestran en la Tabla 2.1.

Tabla 2.1: Middleware Express configurados en app.js

Middleware	Función
<code>cors()</code>	Habilita solicitudes Cross-Origin desde la app Flutter

Middleware	Función
<code>express.json()</code>	Parsea body de requests JSON (límite 10MB)
<code>express.urlencoded()</code>	Parsea body de formularios codificados
<code>express.static()</code>	Sirve imágenes de tickets desde <code>/uploads</code>
<code>logger</code>	Log coloreado de cada request con status code y tiempo
<code>sanitizer</code>	Escapado de caracteres HTML en todos los inputs
<code>apiLimiter</code>	Rate limiting: 200 req/15min para la API general
<code>authLimiter</code>	Rate limiting: 10 req/15min en endpoints de auth
<code>ocrLimiter</code>	Rate limiting: 20 req/15min en escaneo de tickets

2.2. Base de Datos

2.2.1. MySQL 8.0

MySQL 8.0 con motor de almacenamiento InnoDB es el sistema de gestión de base de datos relacional utilizado [4]. InnoDB proporciona soporte de transacciones ACID, claves foráneas y bloqueo a nivel de fila, características esenciales para mantener la integridad de los datos financieros del usuario [5].

El driver de conexión utilizado es **mysql2** con el módulo **promise**, que expone una API basada en **async/await** y utiliza *prepared statements* de forma nativa, previniendo inyecciones SQL [6].

Listing 2.1: Configuración del pool de conexiones MySQL

```

1 // src/config/database.js
2 const mysql = require('mysql2/promise');
3
4 const pool = mysql.createPool({
5   host:           process.env.DB_HOST,
6   port:           parseInt(process.env.DB_PORT) || 3306,
7   user:           process.env.DB_USER,
8   password:       process.env.DB_PASSWORD,
9   database:       process.env.DB_NAME,
10  waitForConnections: true,
11  connectionLimit: 10,      // Maximo 10 conexiones simultaneas
12  timezone:       'local',
13  enableKeepAlive: true,    // Reconexion automatica
14  keepAliveInitialDelay: 30000,
15 });

```

2.3. Dependencias de Producción

La Tabla 2.2 describe todos los paquetes npm declarados en `dependencies` del `package.json`.

Tabla 2.2: Dependencias de producción del backend FINARA

Paquete	Versión	Justificación Técnica
<code>express</code>	4.18.3	Framework web minimalista y maduro; ecosistema de middleware extenso [3]
<code>mysql2</code>	3.9.2	Driver MySQL con prepared statements nativos y API Promise; más rápido que el paquete <code>mysql</code> original [6]
<code>bcrypt</code>	5.1.1	Algoritmo de hash con sal adaptativa para contraseñas; resistente a ataques de fuerza bruta [7]
<code>jsonwebtoken</code>	9.0.2	Implementación del estándar RFC 7519 (JWT) para autenticación sin estado [8]
<code>cors</code>	2.8.5	Middleware CORS con configuración de orígenes permitidos por variable de entorno
<code>dotenv</code>	16.4.5	Carga de variables de entorno desde <code>.env</code> ; evita credenciales en código fuente [9]
<code>multer</code>	1.4.5-lts.1	Middleware multipart/form-data para recepción de imágenes de tickets; límite configurable
<code>express-validator</code>	7.0.1	Librería de validación y sanitización de inputs basada en <code>validator.js</code> [10]
<code>uuid</code>	9.0.1	Generación de identificadores únicos universales v4 para nombrado de archivos subidos

2.4. Dependencias de Desarrollo

Tabla 2.3: Dependencias de desarrollo del backend FINARA

Paquete	Versión	Propósito
---------	---------	-----------

nodemon	3.1.0	Reinicio automático del servidor al detectar cambios en archivos durante desarrollo
jest	29.7.0	Framework de testing con soporte de mocks, cobertura de código y modo watch [11]
supertest	6.3.4	Pruebas de integración HTTP sobre Express sin levantar un servidor real en red [12]

2.5. package.json Completo

Listing 2.2: package.json del backend FINARA v4.0.0

```
1 {
2   "name": "finara-backend",
3   "version": "4.0.0",
4   "description": "API REST - FINARA App de administracion
5     financiera",
6   "main": "server.js",
7   "scripts": {
8     "start": "node server.js",
9     "dev": "nodemon server.js",
10    "test": "jest --coverage --detectOpenHandles",
11    "test:watch": "jest --watch",
12    "test:single": "jest --testPathPattern"
13  },
14  "jest": {
15    "testEnvironment": "node",
16    "testMatch": ["**/tests/**/*.test.js"],
17    "coverageThreshold": {
18      "global": { "functions": 60, "lines": 60 }
19    }
20  },
21  "dependencies": {
22    "bcrypt": "^5.1.1",
23    "cors": "^2.8.5",
24    "dotenv": "^16.4.5",
25    "express": "^4.18.3",
26    "express-validator": "^7.0.1",
27    "jsonwebtoken": "^9.0.2",
28    "multer": "^1.4.5-lts.1",
29    "mysql2": "^3.9.2",
30    "uuid": "^9.0.1"
```

```

30 },
31 "devDependencies": {
32   "jest": "^29.7.0",
33   "nodemon": "^3.1.0",
34   "supertest": "^6.3.4"
35 }
36 }

```

2.6. Stack OCR (Python)

El módulo de reconocimiento óptico de caracteres es un componente externo escrito en Python 3.x, invocado desde Node.js mediante `child_process.spawn`. Las dependencias Python se detallan en la Tabla 2.4.

Tabla 2.4: Dependencias Python del módulo OCR

Librería	Función
opencv-python	Preprocesamiento de imágenes: upscaling adaptativo, filtro bilateral, corrección de iluminación, binarización y sharpening [13]
pytesseract	Wrapper Python para el motor Tesseract OCR de Google; extrae texto con coordenadas y confianza por palabra [14]
numpy	Operaciones matriciales sobre imágenes: kernels de convolución, manipulación de píxeles [15]
difflib	Módulo estándar de Python; usado para fuzzy matching de tiendas sin dependencias externas

2.7. Herramientas de Desarrollo

- **Visual Studio Code:** editor principal con extensiones ESLint, Prettier y REST Client.
- **Postman:** colecciones de prueba de los 46 endpoints con variables de entorno y scripts de test automáticos.
- **MySQL Workbench:** diseño del esquema ER y ejecución del script `schema.sql`.
- **Git:** control de versiones; repositorio privado en GitHub.

- **Overleaf:** compilación y edición de esta documentación LaTeX.

Capítulo 3

Arquitectura del Backend

3.1. Patrón Arquitectónico

El backend de FINARA implementa una **Arquitectura por Capas** inspirada en el patrón MVC (Model-View-Controller), adaptada al contexto de una API REST donde no existe una capa de vistas. Las tres capas del sistema son:

1. **Capa de Enrutamiento (routes/)**: recibe las solicitudes HTTP, aplica middleware de validación y delega al controlador correspondiente. No contiene lógica de negocio.
2. **Capa de Controladores (controllers/)**: orquesta la lógica de negocio. Consulta la base de datos, procesa los datos y construye la respuesta JSON.
3. **Capa de Datos (config/database.js)**: gestiona el pool de conexiones MySQL. Los controladores interactúan directamente con el pool mediante *prepared statements*.

Esta estructura fue elegida por su sencillez y adecuación al tamaño del proyecto. La adición de una capa de servicios o repositorios (*Clean Architecture*) añadiría complejidad sin beneficio significativo para un equipo de dos personas [16].

3.2. Estructura de Carpetas Completa

Listing 3.1: Árbol de directorios del backend FINARA

```
1 finara-backend/  
2   server.js           # Entry point: carga .env,  
                        verifica BD, levanta Express
```

```

3  .env                                # Variables de entorno (no
    versionado en Git)
4  .env.example                       # Plantilla de variables de
    entorno
5  package.json                      # Dependencias y scripts npm
6  README.md                         # Guia de setup y ejecucion
7  database/
8      schema.sql                    # DDL completo: 10 tablas + 2
    triggers
9  ocr/
10     LecturaTickets_v3.py           # Script OCR: pipeline adaptativo
11     categorias.json                # 9 categorias con 200+ palabras
    clave
12     tiendas.json                  # 35 cadenas comerciales con
    variantes OCR
13  src/
14     app.js                         # Configuracion Express:
    middleware, rutas, errores
15     config/
16         database.js                # Pool MySQL con keepAlive y
    reconexion automatica
17         upload.js                  # Configuracion Multer (tipo,
    tamano, destino)
18     controllers/
19         authController.js           # Register, login, logout,
    verify, change-password
20         categoryController.js       # CRUD categorias
21         budgetController.js         # CRUD presupuestos + estado de
    uso
22         incomeController.js         # CRUD ingresos + total mensual
23         expenseController.js        # CRUD gastos + filtros +
    reversion en presupuesto
24         ticketController.js         # OCR scan + CRUD tickets +
    productos
25         debtController.js           # CRUD deudas + registro de pagos
26         alertController.js          # CRUD alertas + config
    notificaciones
27         dashboardController.js      # Resumen financiero + datos
    para graficas
28         userController.js           # Perfil usuario + config
    inicial (onboarding)
29     middleware/
30         auth.js                     # Verificacion JWT en rutas
    protegidas
31         validator.js                # Centralizacion de errores
    express-validator

```

```

32     rateLimiter.js           # Rate limiting por contexto
        (auth, ocr, api)
33     logger.js               # Log coloreado de requests con
        tiempo de respuesta
34     sanitizer.js            # Escapado HTML en todos los
        inputs del body
35     routes/
36     auth.js                 # /api/auth/*
37     categories.js           # /api/categories/*
38     budgets.js              # /api/budgets/*
39     incomes.js              # /api/incomes/*
40     expenses.js             # /api/expenses/*
41     tickets.js              # /api/tickets/*
42     debts.js                # /api/debts/*
43     alerts.js               # /api/alerts/*
44     dashboard.js            # /api/dashboard/*
45     users.js                # /api/users/*
46     services/
47     ocrService.js           # Invocacion del script Python +
        parse de JSON stdout
48     utils/
49     helpers.js              # successResponse,
        errorResponse, suggestCategory, etc.
50     tests/
51     auth.test.js            # 8 casos: register, login,
        verify
52     debts.test.js           # 9 casos: CRUD deudas, pago,
        liquidacion
53     alerts.test.js          # 10 casos: alertas, config
        notificaciones
54     helpers.test.js         # 12 casos: funciones utilitarias
55     middleware.test.js      # 9 casos: sanitizer, auth JWT
56     uploads/                # Imagenes de tickets (creada
        automaticamente)

```

3.3. Descripción de Módulos Principales

Tabla 3.1: Responsabilidad de cada controlador

Controlador	Responsabilidad
authController	Registro con hash bcrypt, login con comparación segura, generación/verificación JWT, cambio de contraseña

Controlador	Responsabilidad
<code>categoryController</code>	CRUD de categorías predefinidas y personalizadas; bloqueo de eliminación si tienen gastos vinculados
<code>budgetController</code>	CRUD de presupuestos con cálculo de porcentaje utilizado; consulta por categoría
<code>incomeController</code>	CRUD de ingresos con filtro mensual y cálculo de total por período
<code>expenseController</code>	CRUD de gastos con reversión automática de <code>monto_utilizado</code> al eliminar; filtros por mes, categoría y recientes
<code>ticketController</code>	Recibe imágenes multipart, invoca <code>ocrService</code> , crea Ticket + Productos_Tickets + Gasto de forma atómica
<code>debtController</code>	CRUD de deudas con cálculo de <code>porcentaje_pagado</code> ; registro de pagos con avance de fecha y detección de liquidación
<code>alertController</code>	CRUD de alertas; marcar como leída (individual y masiva); CRUD de config de notificaciones
<code>dashboardController</code>	Resumen financiero mensual (ingresos, gastos, balance, top categorías); datos para gráficas de gastos y presupuestos
<code>userController</code>	Perfil de usuario; actualización de flags de onboarding; configuración inicial (modo, distribución, ingreso base)

3.4. Flujo de una Solicitud HTTP

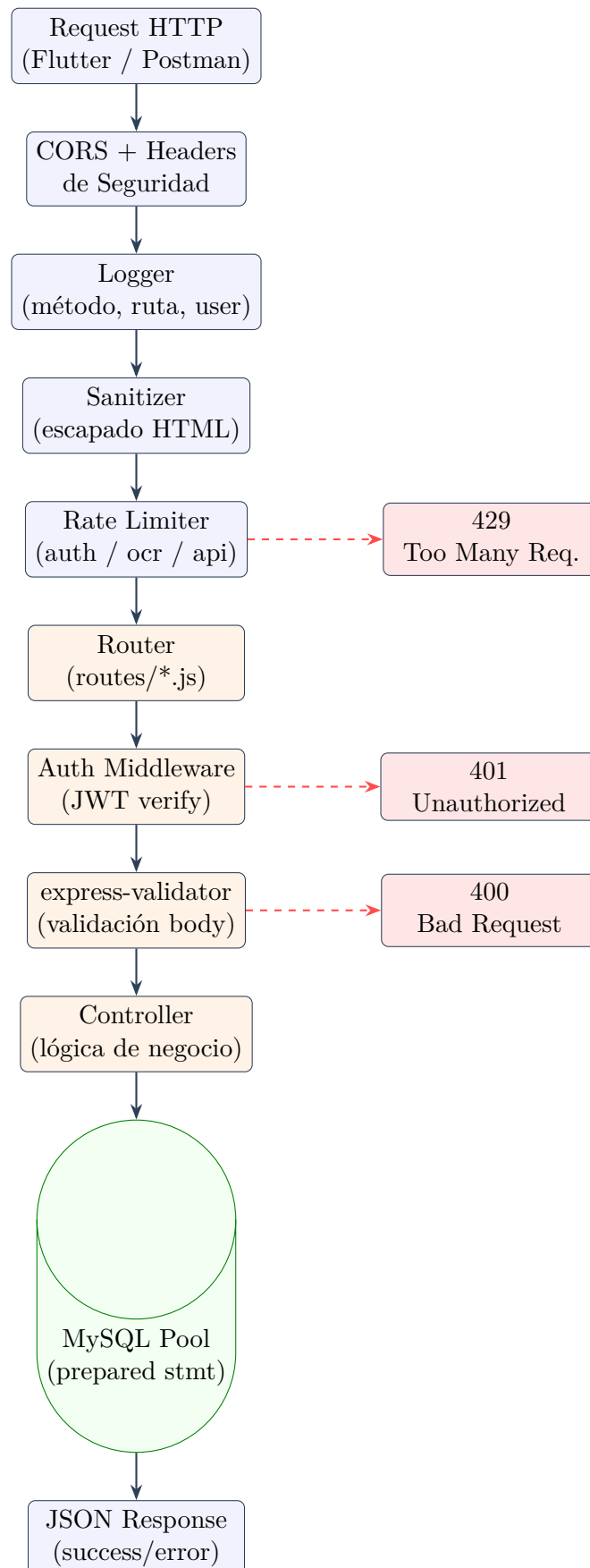


Figura 3.1: Flujo de procesamiento de una solicitud HTTP en FINARA Backend

3.5. Formato Estándar de Respuestas

Todos los endpoints de la API retornan JSON con la misma estructura, diferenciando entre éxito y error mediante el campo `success`.

Listing 3.2: Formatos de respuesta estándar de la API

```
1 // Respuesta exitosa
2 {
3   "success": true,
4   "data": { ... } // objeto o arreglo segun el endpoint
5 }
6
7 // Respuesta de error
8 {
9   "success": false,
10  "error": "Mensaje descriptivo del error"
11 }
12
13 // Error de validacion (400)
14 {
15   "success": false,
16   "error": "El primer mensaje de error",
17   "errors": [
18     { "field": "email", "msg": "Email invalido" },
19     { "field": "password", "msg": "Minimo 6 caracteres" }
20   ]
21 }
```

3.6. Flujo Especial: Escaneo OCR de Ticket

El flujo del endpoint POST `/api/tickets/scan` involucra componentes adicionales:

Listing 3.3: Flujo completo de escaneo de ticket con OCR

```
1 // 1. Multer recibe las imagenes (multipart/form-data)
2 //   Valida tipo (image/*) y tamaño (max 10MB)
3 //   Guarda en: uploads/<uuid>.jpg
4
5 // 2. ticketController.scan() invoca ocrService
6 const ocrResult = await processMultipleImages(uploadedPaths);
7
8 // 3. ocrService.js lanza el proceso Python
9 const proc = spawn('python3', ['ocr/LecturaTickets_v3.py',
10   imagePath]);
11 // El script Python imprime JSON puro por stdout
```

```
11 // Los logs de debug van por stderr (invisibles para Node.js)
12
13 // 4. Python retorna JSON estructurado:
14 // { success, store_name, total, confidence, products: [...] }
15
16 // 5. ticketController inserta atómicamente:
17 //     INSERT INTO Tickets (...)
18 //     INSERT INTO Productos_Tickets (...) -- uno por producto
19 //     INSERT INTO Gastos (...)           -- gasto automatico
    vinculado
20
21 // 6. Los triggers MySQL se disparan automáticamente:
22 //     - actualizar_presupuesto_gasto: actualiza monto_utilizado
23 //     - verificar_presupuesto_excedido: crea Alerta si >= 90%
```

Capítulo 4

Documentación de Endpoints

La API REST de FINARA expone **46 endpoints** organizados en 10 módulos funcionales. Todos los endpoints (excepto `/api/health`, `/api/auth/register` y `/api/auth/login`) requieren autenticación mediante token JWT en el header `Authorization: Bearer <token>`.

Tabla 4.1: Resumen de todos los endpoints del backend FINARA

Módulo	Prefijo	Descripción	Auth	Cant.
Health	<code>/api/health</code>	Estado del servidor	No	1
Auth	<code>/api/auth</code>	Autenticación y sesión	Mixto	5
Users	<code>/api/users</code>	Perfil y configuración inicial	Sí	5
Categories	<code>/api/categories</code>	Categorías de gastos	Sí	5
Budgets	<code>/api/budgets</code>	Presupuestos mensuales	Sí	6
Incomes	<code>/api/incomes</code>	Ingresos del usuario	Sí	6
Expenses	<code>/api/expenses</code>	Gastos registrados	Sí	7
Tickets	<code>/api/tickets</code>	Tickets OCR	Sí	5
Debts	<code>/api/debts</code>	Deudas y pagos	Sí	6
Alerts	<code>/api/alerts</code>	Alertas y notificaciones	Sí	7
Dashboard	<code>/api/dashboard</code>	Resumen financiero	Sí	3
			Total	46

4.1. Autenticación (/api/auth)

4.1.1. POST /api/auth/register

POST /api/auth/register — Registrar usuario

Descripción: Crea un nuevo usuario en el sistema. Al completarse, genera automáticamente 10 categorías de gasto predefinidas y una configuración de notificaciones para el usuario.

Auth requerida: No

Request Body (JSON):

```
1 {
2   "nombre":    "string  (requerido, min 2 chars)",
3   "email":     "string  (requerido, formato email valido)",
4   "password":  "string  (requerido, min 6 caracteres)"
5 }
```

Validaciones:

- nombre: no vacío, mínimo 2 caracteres.
- email: formato válido RFC 5322, debe ser único en la BD.
- password: mínimo 6 caracteres; se almacena como hash bcrypt (10 rounds).

Response 201 (Created):

```
1 {
2   "success": true,
3   "data": {
4     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
5     "user": {
6       "id": 1,
7       "nombre": "Juan Perez",
8       "email": "juan@email.com",
9       "onboarding_completado": false,
10      "setup_completado": false
11    }
12  }
13 }
```

Responses de Error:

- 400 — Campos faltantes o inválidos (mensaje de validación).
- 400 — El email ya está registrado.
- 500 — Error interno del servidor.

4.1.2. POST /api/auth/login

POST /api/auth/login — Iniciar sesión

Descripción: Autentica un usuario existente. Retorna el token JWT y los flags de estado del onboarding para que la app móvil determine a qué pantalla navegar.

Request Body:

```
1 {
2   "email":      "string (requerido)",
3   "password":  "string (requerido)"
4 }
```

Response 200:

```
1 {
2   "success": true,
3   "data": {
4     "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
5     "user": {
6       "id": 1,
7       "nombre": "Juan Perez",
8       "email": "juan@email.com",
9       "onboarding_completado": true,
10      "setup_completado": true
11    }
12  }
13 }
```

Errors: 400 campos faltantes | 401 credenciales incorrectas.

4.1.3. POST /api/auth/logout

POST /api/auth/logout — Cerrar sesión

Endpoint de confirmación de cierre de sesión. La invalidación real del JWT se maneja del lado del cliente eliminando el token almacenado.

Auth: Sí | **Response 200:** {"success": true, "data": {"message": "Sesión cerrada"}}

4.1.4. GET /api/auth/verify

GET /api/auth/verify — Verificar token

Verifica la validez del JWT enviado en el header. Útil para que la app verifique sesión al iniciar.

Response 200: datos del usuario decodificados del token. **Error 401:** token expirado o inválido.

4.1.5. PUT /api/auth/change-password

PUT /api/auth/change-password — Cambiar contraseña

Request Body:

```
1 {  
2   "currentPassword": "string (requerido)",  
3   "newPassword":    "string (min 6 caracteres)"  
4 }
```

Errors: 400 contraseña actual incorrecta | 400 nueva contraseña inválida.

4.2. Usuarios (/api/users)

4.2.1. GET /api/users/me

Retorna el perfil completo del usuario autenticado: id, nombre, email, onboarding_completado, setup_completado, fecha_creacion.

4.2.2. PUT /api/users/me

Actualiza campos del perfil. Body opcional con cualquier combinación de:

```
1 {  
2   "nombre":           "string?",  
3   "onboarding_completado": "boolean?",  
4   "setup_completado":   "boolean?"  
5 }
```

4.2.3. DELETE /api/users/me

Elimina la cuenta y todos sus datos en cascada. Acción irreversible.

4.2.4. POST /api/users/initial-config

Guarda la configuración inicial del onboarding. Marca `setup_completado = true` automáticamente. Si se provee `ingreso_mensual_inicial`, crea un ingreso recurrente en la tabla Ingresos.

```
1 {
2   "modo_administracion": "'unico' | 'multiple'",
3   "ingreso_mensual_inicial": 15000.00,
4   "distribucion_categorias": {
5     "Alimentos": 40, "Transporte": 20, "Otros": 40
6   }
7 }
```

4.2.5. GET /api/users/initial-config

Retorna la configuración inicial guardada, con `distribucion_categorias` ya parseada como objeto JSON.

4.3. Categorías (/api/categories)

4.3.1. GET /api/categories

Lista todas las categorías del usuario (predefinidas + personalizadas), ordenadas por tipo y nombre.

Response 200:

```
1 {
2   "success": true,
3   "data": [
4     {
5       "id_categoria": 1,
6       "nombre_categoria": "Alimentos",
7       "tipo_categoria": "predefinida",
8       "icono": "restaurant",
9       "color": "#FF6B6B"
10    }
11  ]
12 }
```

```
11 ]  
12 }
```

4.3.2. GET /api/categories/:id

Obtiene una categoría por ID. Retorna 404 si no pertenece al usuario.

4.3.3. POST /api/categories

Crea categoría personalizada. `tipo_categoria` se establece automáticamente como "personalizada".

```
1 { "nombre_categoria": "Mascotas", "icono": "pets", "color":  
  "#4CAF50" }
```

4.3.4. PUT /api/categories/:id

Actualiza nombre, icono o color. Body parcial aceptado.

4.3.5. DELETE /api/categories/:id

Elimina categoría. Bloquea con 400 si:

- La categoría es de tipo "predefinida".
- Tiene gastos vinculados (restricción de clave foránea).

4.4. Presupuestos (/api/budgets)

4.4.1. GET /api/budgets

Lista todos los presupuestos activos del usuario. Incluye campos calculados: `porcentaje_utilizado` y `monto_disponible`.

4.4.2. GET /api/budgets/category/:categoryId

Obtiene el presupuesto activo para una categoría específica. Útil para mostrar el estado del presupuesto al registrar un gasto.

4.4.3. POST /api/budgets

```
1 {  
2   "id_categoria": 1,  
3   "monto_asignado": 3000.00,  
4   "periodo": "mensual",  
5   "fecha_inicio": "2025-03-01",  
6   "fecha_fin": "2025-03-31"  
7 }
```

Validaciones: monto_asignado > 0; periodo en [mensual, semanal, anual]; fechas válidas.

4.4.4. PUT /api/budgets/:id

Actualiza cualquier campo. Body parcial aceptado con COALESCE.

4.4.5. DELETE /api/budgets/:id

Elimina presupuesto. 404 si no pertenece al usuario.

4.5. Ingresos (/api/incomes)

4.5.1. GET /api/incomes

Lista ingresos con filtro opcional de período:

```
1 GET /api/incomes?month=3&year=2025
```

4.5.2. GET /api/incomes/total

Retorna la suma de todos los ingresos del mes especificado.

```
1 { "success": true, "data": { "total": 15000.00, "month": 3,  
   "year": 2025 } }
```

4.5.3. POST /api/incomes

```
1 {  
2   "nombre_ingreso": "Salario mensual",  
3   "monto":         15000.00,  
4   "es_recurrente": true,  
5   "fecha":         "2025-03-01"  
6 }
```

4.5.4. PUT /api/incomes/:id y DELETE /api/incomes/:id

CRUD estándar. Body parcial en PUT.

4.6. Gastos (/api/expenses)

4.6.1. GET /api/expenses

Lista gastos con filtro de mes/año. Incluye nombre de categoría en cada elemento.

4.6.2. GET /api/expenses/recent

Últimos gastos registrados. Query param ?limit=10 (default 10, máx 50).

4.6.3. GET /api/expenses/category/:categoryId

Filtra gastos por categoría del mes actual.

4.6.4. POST /api/expenses

Crea gasto manual. El trigger MySQL actualizar_presupuesto_gasto actualiza automáticamente el monto_utilizado del presupuesto activo de esa categoría.

```
1 {  
2   "id_categoria": 1,  
3   "nombre_gasto": "Super del mes",  
4   "monto":       850.50,  
5   "fecha":       "2025-03-15",  
6   "notas":       "Compra semanal (opcional)"  
7 }
```

4.6.5. DELETE /api/expenses/:id

Elimina el gasto y **revierte** el monto_utilizado del presupuesto correspondiente, si existe.

4.7. Tickets OCR (/api/tickets)

4.7.1. POST /api/tickets/scan

POST /api/tickets/scan — Escanear ticket con OCR

Content-Type: multipart/form-data

Campo: images (File, máximo 5 imágenes, máximo 10MB por archivo)

Descripción: Recibe imágenes de ticket, las procesa con el pipeline OCR Python/Tesseract y guarda atómicamente en la BD: el ticket, sus productos y un gasto vinculado.

Response 201:

```
1 {
2   "success": true,
3   "data": {
4     "ticket": {
5       "id": 1,
6       "nombre_tienda": "OXXO",
7       "total": 122.94,
8       "confidence": 0.627,
9       "fecha_compra": "2025-10-04T00:00:00.000Z"
10    },
11    "productos": [
12      {
13        "name": "POWERADE",
14        "quantity": 1,
15        "unit_price": 41.00,
16        "total": 41.00,
17        "categoria": "Alimentos"
18      }
19    ],
20    "gasto": {
21      "id": 1,
22      "monto": 122.94,
23      "categoria": "Alimentos",
24      "id_categoria": 2
25    }
26  }
```

```
27 }
```

Errors: 400 sin imágenes | 429 límite OCR excedido | 500 error del script Python.

4.7.2. GET /api/tickets

Lista todos los tickets del usuario con conteo de productos (`total_productos`).

4.7.3. GET /api/tickets/:id

Ticket individual con rutas de imágenes como arreglo JSON.

4.7.4. GET /api/tickets/:id/products

Lista los productos extraídos de un ticket con categoría, icono y color.

4.7.5. DELETE /api/tickets/:id

Elimina el ticket y sus archivos de imagen del disco.

4.8. Deudas (/api/debts)

4.8.1. GET /api/debts

Lista deudas ordenadas: activas primero, luego por próxima fecha de pago. Incluye campos calculados: `porcentaje_pagado`, `monto_pendiente`, `dias_para_pago`.

4.8.2. POST /api/debts

```
1 {  
2   "nombre_deuda":      "Tarjeta Banamex",  
3   "monto_total":      12000.00,  
4   "pago_mensual":     1000.00,  
5   "duracion_meses":   12,  
6   "fecha_siguiente_pago": "2025-04-01"  
7 }
```

4.8.3. PUT /api/debts/:id/pay -- Registrar Pago

PUT /api/debts/:id/pay — Registrar pago de deuda

Registra un pago mensual. Si no se provee monto_pago, usa el pago_mensual configurado. Avanza fecha_siguiete_pago un mes. Si monto_pagado $\geq$ monto_total, marca activa = false.

Request Body:

```
1 { "monto_pago": 1000.00 }
```

Response 200:

```
1 {
2   "success": true,
3   "data": {
4     "deuda": { "id_deuda": 1, "monto_pagado": 2000.00,
5               "porcentaje_pagado": 16.7, "activa": true },
6     "liquidada": false,
7     "monto_pagado": 1000.00,
8     "message": "Pago de $1000 registrado correctamente"
9   }
}
```

4.9. Alertas (/api/alerts)

4.9.1. GET /api/alerts/unread

Retorna el conteo y lista de alertas no leídas. Usado por la app para mostrar el badge de notificaciones.

```
1 { "success": true, "data": { "count": 3, "alertas": [...] } }
```

4.9.2. PUT /api/alerts/mark-all-read

Marca todas las alertas del usuario como leídas en una sola operación.

4.9.3. GET /api/alerts/config y PUT /api/alerts/config

Obtiene y actualiza la configuración de notificaciones:

```
1 {
```

```
2  "presupuesto_excedido": true,  
3  "recordatorio_gestion": true,  
4  "pago_deuda_proximo": false  
5  }
```

4.10. Dashboard (/api/dashboard)

4.10.1. GET /api/dashboard/summary

Resumen financiero del mes. Query params: ?month=3&year=2025.

```
1  {  
2    "success": true,  
3    "data": {  
4      "periodo": { "month": 3, "year": 2025 },  
5      "ingresos": 15000.00,  
6      "gastos": 8500.00,  
7      "balance": 6500.00,  
8      "top_categorias": [  
9        { "nombre_categoria": "Alimentos", "total": 3200.00 }  
10     ]  
11   }  
12 }
```

4.10.2. GET /api/dashboard/charts/expenses

Datos para gráfica de pastel de gastos por categoría del mes. Incluye nombre, total y porcentaje de cada categoría.

4.10.3. GET /api/dashboard/charts/budgets

Estado de todos los presupuestos activos: monto asignado, utilizado, disponible y porcentaje para gráfica de barras de progreso.

Capítulo 5

Base de Datos

5.1. Visión General

La base de datos **finara_db** está implementada en MySQL 8.0 con motor InnoDB. Consta de **10 tablas**, relaciones con integridad referencial mediante claves foráneas, 2 triggers automáticos y zona horaria configurada en la conexión del pool.

Tabla 5.1: Tablas de la base de datos finara_db

Tabla	Filas aprox.	Propósito
Usuarios	1–100	Cuentas de usuario con flags de onboarding
Categorias	10–50	Categorías de gasto predefinidas y personalizadas
Presupuestos	0–20	Presupuestos mensuales por categoría
Ingresos	1–500	Registro de ingresos del usuario
Gastos	0–5000	Gastos manuales y generados por OCR
Tickets	0–1000	Tickets escaneados con metadatos OCR
Productos_Tickets	0–10000	Ítems extraídos de cada ticket
Deudas	0–20	Deudas con seguimiento de pagos
Alertas	0–200	Notificaciones automáticas del sistema
Configuracion_Notif.	1	Preferencias de notificaciones por usuario
Configuracion_Inicial	1	Configuración del onboarding inicial

5.2. Descripción de Tablas

5.2.1. Tabla: Usuarios

Listing 5.1: DDL de la tabla Usuarios

```

1 CREATE TABLE Usuarios (
2   id_usuario          INT          PRIMARY KEY AUTO_INCREMENT,
3   nombre              VARCHAR(100) NOT NULL,
4   email               VARCHAR(100) UNIQUE NOT NULL,
5   password_hash       VARCHAR(255) NOT NULL,
6   onboarding_completado BOOLEAN    DEFAULT FALSE,
7   setup_completado    BOOLEAN    DEFAULT FALSE,
8   fecha_creacion      TIMESTAMP    DEFAULT CURRENT_TIMESTAMP
9 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Tabla 5.2: Columnas de la tabla Usuarios

Columna	Tipo	Descripción
id_usuario	INT PK AI	Identificador único autoincrementable
nombre	VARCHAR(100)	Nombre completo del usuario
email	VARCHAR(100) UNIQUE	Email de acceso; índice único
password_hash	VARCHAR(255)	Hash bcrypt de la contraseña (10 rounds)
onboarding_completado	BOOLEAN	TRUE cuando el usuario completó el flujo de bienvenida
setup_completado	BOOLEAN	TRUE cuando guardó la configuración inicial
fecha_creacion	TIMESTAMP	Fecha y hora de registro

5.2.2. Tabla: Categorías

Listing 5.2: DDL de la tabla Categorías

```

1 CREATE TABLE Categorías (
2   id_categoria      INT          PRIMARY KEY AUTO_INCREMENT,
3   id_usuario        INT          NOT NULL,
4   nombre_categoria  VARCHAR(50) NOT NULL,
5   tipo_categoria    ENUM('predefinida','personalizada') DEFAULT
6   'personalizada',
7   icono             VARCHAR(50)  DEFAULT 'category',
8   color             VARCHAR(7)   DEFAULT '#6B7280',

```

```

8  FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario) ON
    DELETE CASCADE
9 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Las 10 categorías predefinidas creadas automáticamente al registrar un usuario son: Alimentos, Transporte, Salud, Entretenimiento, Servicios, Ropa, Hogar, Educación, Restaurantes y Otros.

5.2.3. Tabla: Presupuestos

Listing 5.3: DDL de la tabla Presupuestos

```

1 CREATE TABLE Presupuestos (
2   id_presupuesto INT PRIMARY KEY AUTO_INCREMENT,
3   id_usuario INT NOT NULL,
4   id_categoria INT NOT NULL,
5   monto_asignado DECIMAL(10,2) NOT NULL,
6   monto_utilizado DECIMAL(10,2) DEFAULT 0.00,
7   periodo ENUM('mensual','semanal','anual') DEFAULT
    'mensual',
8   fecha_inicio DATE NOT NULL,
9   fecha_fin DATE NOT NULL,
10  activo BOOLEAN DEFAULT TRUE,
11  FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario) ON
    DELETE CASCADE,
12  FOREIGN KEY (id_categoria) REFERENCES Categorias(id_categoria)
13 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

5.2.4. Tabla: Ingresos

Listing 5.4: DDL de la tabla Ingresos

```

1 CREATE TABLE Ingresos (
2   id_ingreso INT PRIMARY KEY AUTO_INCREMENT,
3   id_usuario INT NOT NULL,
4   nombre_ingreso VARCHAR(100) NOT NULL,
5   monto DECIMAL(10,2) NOT NULL,
6   es_recurrente BOOLEAN DEFAULT FALSE,
7   fecha DATE NOT NULL,
8   fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
9   FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario) ON
    DELETE CASCADE
10 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

5.2.5. Tabla: Gastos

Listing 5.5: DDL de la tabla Gastos

```

1 CREATE TABLE Gastos (
2   id_gasto          INT          PRIMARY KEY AUTO_INCREMENT,
3   id_usuario        INT          NOT NULL,
4   id_ticket         INT          NULL,      -- NULL si es gasto manual
5   id_categoria      INT          NOT NULL,
6   nombre_gasto      VARCHAR(100) NOT NULL,
7   monto             DECIMAL(10,2) NOT NULL,
8   fecha             DATE         NOT NULL,
9   extraido_por_ocr  BOOLEAN      DEFAULT FALSE,
10  notas             TEXT         NULL,
11  fecha_creacion    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
12  FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario) ON
    DELETE CASCADE,
13  FOREIGN KEY (id_ticket) REFERENCES Tickets(id_ticket) ON
    DELETE SET NULL,
14  FOREIGN KEY (id_categoria) REFERENCES Categorias(id_categoria)
15 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

5.2.6. Tabla: Tickets

Listing 5.6: DDL de la tabla Tickets

```

1 CREATE TABLE Tickets (
2   id_ticket         INT          PRIMARY KEY AUTO_INCREMENT,
3   id_usuario        INT          NOT NULL,
4   nombre_tienda     VARCHAR(100) DEFAULT 'Desconocida',
5   total             DECIMAL(10,2) DEFAULT 0.00,
6   metodo_entrada    ENUM('ocr','manual') DEFAULT 'ocr',
7   confianza_ocr      DECIMAL(4,3) NULL,      -- 0.000 a 1.000
8   rutas_imagenes    JSON        NULL,      -- arreglo de rutas de
    archivo
9   fecha_compra      DATE         NOT NULL,
10  fecha_creacion    TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
11  FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario) ON
    DELETE CASCADE
12 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

5.2.7. Tabla: Productos_Tickets

Listing 5.7: DDL de la tabla Productos_Tickets

```
1 CREATE TABLE Productos_Tickets (  
2     id_producto      INT          PRIMARY KEY AUTO_INCREMENT,  
3     id_ticket        INT          NOT NULL,  
4     id_categoria     INT          NULL,  
5     nombre_producto  VARCHAR(150) NOT NULL,  
6     precio           DECIMAL(10,2) NOT NULL,  
7     FOREIGN KEY (id_ticket) REFERENCES Tickets(id_ticket)  
        ON DELETE CASCADE,  
8     FOREIGN KEY (id_categoria) REFERENCES Categorias(id_categoria)  
        ON DELETE SET NULL  
9 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.2.8. Tabla: Deudas

Listing 5.8: DDL de la tabla Deudas

```
1 CREATE TABLE Deudas (  
2     id_deuda          INT          PRIMARY KEY AUTO_INCREMENT,  
3     id_usuario        INT          NOT NULL,  
4     nombre_deuda      VARCHAR(100) NOT NULL,  
5     monto_total       DECIMAL(10,2) NOT NULL,  
6     monto_pagado      DECIMAL(10,2) DEFAULT 0.00,  
7     pago_mensual      DECIMAL(10,2) NOT NULL,  
8     duracion_meses    INT          NOT NULL,  
9     fecha_siguiente_pago DATE      NOT NULL,  
10    activa            BOOLEAN      DEFAULT TRUE,  
11    fecha_creacion     TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,  
12    FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario) ON  
        DELETE CASCADE  
13 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.2.9. Tabla: Alertas

Listing 5.9: DDL de la tabla Alertas

```
1 CREATE TABLE Alertas (  
2     id_alerta      INT          PRIMARY KEY AUTO_INCREMENT,  
3     id_usuario     INT          NOT NULL,  
4     tipo_alerta    VARCHAR(50) NOT NULL, -- 'presupuesto', 'deuda',  
        'gestion'  
5     titulo        VARCHAR(100) NOT NULL,  
6     mensaje       TEXT          NOT NULL,  
7     leida         BOOLEAN      DEFAULT FALSE,  
8     fecha         TIMESTAMP    DEFAULT CURRENT_TIMESTAMP,
```

```
9 FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario) ON
DELETE CASCADE
10 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.2.10. Tabla: Configuracion_Notificaciones

Listing 5.10: DDL de Configuracion_Notificaciones

```
1 CREATE TABLE Configuracion_Notificaciones (
2   id_config          INT          PRIMARY KEY AUTO_INCREMENT,
3   id_usuario         INT          UNIQUE NOT NULL,
4   presupuesto_excedido BOOLEAN DEFAULT TRUE,
5   recordatorio_gestion BOOLEAN DEFAULT TRUE,
6   pago_deuda_proximo BOOLEAN DEFAULT TRUE,
7   FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario) ON
DELETE CASCADE
8 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.2.11. Tabla: Configuracion_Inicial

Listing 5.11: DDL de Configuracion_Inicial

```
1 CREATE TABLE Configuracion_Inicial (
2   id_config          INT          PRIMARY KEY AUTO_INCREMENT,
3   id_usuario         INT          UNIQUE NOT NULL,
4   modo_administracion ENUM('unico','multiple') DEFAULT 'unico',
5   distribucion_categorias JSON      NULL,
6   ingreso_mensual_inicial DECIMAL(10,2) NULL,
7   FOREIGN KEY (id_usuario) REFERENCES Usuarios(id_usuario) ON
DELETE CASCADE
8 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

5.3. Triggers Automáticos

El sistema utiliza dos triggers MySQL que implementan lógica de negocio a nivel de base de datos, garantizando la integridad incluso si se insertan datos por fuera de la API.

5.3.1. Trigger: actualizar_presupuesto_gasto

Listing 5.12: Trigger de actualización de presupuesto al registrar gasto

```
1 DELIMITER $$
2 CREATE TRIGGER actualizar_presupuesto_gasto
3 AFTER INSERT ON Gastos
4 FOR EACH ROW
5 BEGIN
6     UPDATE Presupuestos
7     SET monto_utilizado = monto_utilizado + NEW.monto
8     WHERE id_usuario = NEW.id_usuario
9     AND id_categoria = NEW.id_categoria
10    AND activo = TRUE
11    AND fecha_inicio <= NEW.fecha
12    AND fecha_fin >= NEW.fecha;
13 END$$
14 DELIMITER ;
```

5.3.2. Trigger: verificar_presupuesto_excedido

Listing 5.13: Trigger de alerta cuando el presupuesto alcanza el 90 %

```
1 DELIMITER $$
2 CREATE TRIGGER verificar_presupuesto_excedido
3 AFTER UPDATE ON Presupuestos
4 FOR EACH ROW
5 BEGIN
6     IF NEW.monto_utilizado >= (NEW.monto_asignado * 0.90) THEN
7         INSERT INTO Alertas (id_usuario, tipo_alerta, titulo, mensaje)
8         VALUES (
9             NEW.id_usuario,
10            'presupuesto',
11            'Presupuesto al limite',
12            CONCAT('Tu presupuesto ha alcanzado el ',
13                ROUND((NEW.monto_utilizado/NEW.monto_asignado)*100,
14                    0),
15                '% del limite asignado.')
```

5.4. Diagrama Entidad-Relación

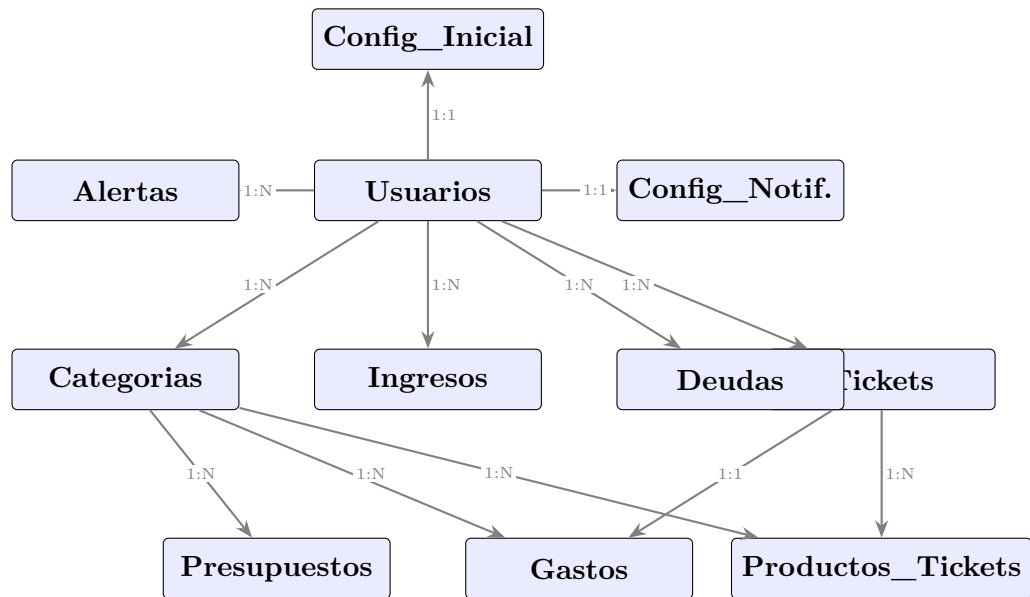


Figura 5.1: Diagrama Entidad-Relación simplificado de finara_db

Capítulo 6

Autenticación y Seguridad

6.1. Estrategia de Autenticación

FINARA implementa autenticación **stateless** mediante **JSON Web Tokens (JWT)**, estándar definido en el RFC 7519 [8]. Esta estrategia elimina la necesidad de almacenar sesiones en el servidor, simplificando el escalado horizontal y la comunicación con la app móvil.

6.1.1. JSON Web Token (JWT)

El token generado al hacer login contiene tres secciones codificadas en Base64URL separadas por puntos:

Listing 6.1: Estructura del JWT generado por FINARA

```
1 // Header
2 { "alg": "HS256", "typ": "JWT" }
3
4 // Payload (claims)
5 {
6   "userId": 1,
7   "email": "juan@email.com",
8   "iat": 1741000000, // issued at (Unix timestamp)
9   "exp": 1741604800 // expires at (+7 dias)
10 }
11
12 // Firma: HMACSHA256(base64(header) + "." + base64(payload),
13   JWT_SECRET)
```

Listing 6.2: Generación del token JWT en authController.js

```
1 const jwt = require('jsonwebtoken');
2
3 // Al hacer login o register exitoso:
4 const token = jwt.sign(
5   { userId: user.id_usuario, email: user.email },
6   process.env.JWT_SECRET,           // Almacenado en .env, nunca en
   código
7   { expiresIn: process.env.JWT_EXPIRES_IN || '7d' }
8 );
```

6.1.2. Hash de Contraseñas con bcrypt

Las contraseñas se almacenan exclusivamente como hash bcrypt con **10 rounds de sal** [7]. El factor de costo 10 implica $2^{10} = 1024$ iteraciones del algoritmo, haciendo impráctico el ataque por fuerza bruta.

Listing 6.3: Hash y verificación de contraseña con bcrypt

```
1 const bcrypt = require('bcrypt');
2 const SALT_ROUNDS = 10;
3
4 // Al registrar usuario:
5 const password_hash = await bcrypt.hash(password, SALT_ROUNDS);
6
7 // Al hacer login:
8 const esValida = await bcrypt.compare(passwordIngresada,
   user.password_hash);
9 if (!esValida) return ErrorResponse(res, 'Credenciales
   incorrectas', 401);
```

6.2. Middleware de Autenticación

Todas las rutas protegidas pasan por `src/middleware/auth.js` antes de llegar al controlador.

Listing 6.4: Middleware de autenticación JWT completo

```
1 // src/middleware/auth.js
2 const jwt = require('jsonwebtoken');
3
4 const authMiddleware = (req, res, next) => {
5   try {
6     const authHeader = req.headers['authorization'];
7
```

```
8     if (!authHeader || !authHeader.startsWith('Bearer ')) {
9         return res.status(401).json({
10             success: false,
11             error: 'Token de autenticacion requerido',
12         });
13     }
14
15     const token = authHeader.split(' ')[1];
16     const decoded = jwt.verify(token, process.env.JWT_SECRET);
17
18     // Adjunta datos del usuario al objeto request
19     req.user = { userId: decoded.userId, email: decoded.email };
20
21     next();
22 } catch (error) {
23     if (error.name === 'TokenExpiredError') {
24         return res.status(401).json({
25             success: false,
26             error: 'Token expirado, inicia sesion nuevamente',
27         });
28     }
29     return res.status(401).json({ success: false, error: 'Token
30         invalido' });
31 };
32
33 module.exports = authMiddleware;
```

6.3. Validación de Inputs con express-validator

Cada endpoint con datos de entrada implementa validación mediante `express-validator` [10]. Las validaciones se declaran en la definición de rutas y son procesadas por el middleware centralizado `validator.js`.

Listing 6.5: Validaciones del endpoint de registro

```
1 // src/routes/auth.js
2 const { body } = require('express-validator');
3
4 router.post('/register',
5     [
6         body('nombre')
7             .trim()
8             .notEmpty().withMessage('El nombre es requerido')
9             .isLength({ min: 2 }).withMessage('Minimo 2 caracteres'),
```

```
10     body('email')
11       .isEmail().withMessage('Email invalido')
12       .normalizeEmail(),
13     body('password')
14       .isLength({ min: 6 }).withMessage('La contraseña debe tener
           minimo 6 caracteres'),
15   ],
16   validate,          // middleware centralizado
17   register           // controlador
18 );
```

6.4. Sanitización de Inputs

El middleware `sanitizer.js` escapa caracteres HTML en todos los campos de texto del body antes de que lleguen al controlador, previniendo ataques XSS [17].

Listing 6.6: Middleware sanitizador de inputs

```
1 // src/middleware/sanitizer.js
2 const sanitizeString = (str) => {
3   if (typeof str !== 'string') return str;
4   return str
5     .replace(/</g, '&lt;')
6     .replace(/"/g, '&quot;')
7     .trim();
8 };
9
10 const sanitizer = (req, res, next) => {
11   if (req.body && typeof req.body === 'object') {
12     req.body = sanitizeObject(req.body);
13   }
14   next();
15 };
```

6.5. Rate Limiting

El middleware `rateLimiter.js` implementa limitación de tasa de solicitudes para prevenir ataques de fuerza bruta y abuso de la API [18]. Usa un Map en memoria (adecuado para una sola instancia en desarrollo).

Tabla 6.1: Configuración de rate limiting por contexto

Contexto	Máx. Solicitudes	Ventana	Justificación
/api/auth/*	10	15 min	Previene fuerza bruta en login
/api/tickets/scan	20	15 min	OCR es costoso en CPU/- tiempo
/api/* (general)	200	15 min	Uso normal de la API

6.6. Headers de Seguridad

En lugar de añadir la dependencia `helmet`, los headers de seguridad se configuran manualmente en `app.js` [19]:

Listing 6.7: Headers de seguridad HTTP configurados manualmente

```

1 app.use((req, res, next) => {
2   res.setHeader('X-Content-Type-Options', 'nosniff');
3   // Previene MIME sniffing
4
5   res.setHeader('X-Frame-Options', 'DENY');
6   // Previene clickjacking (no iframes)
7
8   res.setHeader('X-XSS-Protection', '1; mode=block');
9   // Activa filtro XSS del navegador
10
11  res.setHeader('Referrer-Policy', 'no-referrer');
12  // No envia Referer en solicitudes cross-origin
13
14  next();
15 });

```

6.7. Prevención de Inyección SQL

El driver `mysql2` utiliza **prepared statements** de forma nativa, enviando los parámetros separados de la consulta SQL. Esto hace imposible la inyección SQL independientemente del valor del input [20].

Listing 6.8: Prepared statement con `mysql2`

```

1 // Correcto: parametro separado de la consulta
2 const [rows] = await pool.query(
3   'SELECT * FROM Usuarios WHERE email = ?',

```

```

4   [email]    // <- mysql2 envia esto como parametro, nunca
        interpolado
5   );
6
7   // Incorrecto (no se hace en FINARA):
8   // pool.query('SELECT * FROM Usuarios WHERE email = '${email}')
```

6.8. Resumen de Medidas de Seguridad

Tabla 6.2: Resumen de medidas de seguridad implementadas

Amenaza	Mitigación	Implementación
Acceso no autorizado	JWT + expiración 7d	middleware/auth.js
Contraseñas robadas	bcrypt 10 rounds	authController.js
Fuerza bruta login	Rate limit 10/15min	middleware/rateLimiter.js
XSS en inputs	Escapado HTML	middleware/sanitizer.js
Inyección SQL	Prepared statements	mysql2 driver
Clickjacking	X-Frame-Options	app.js headers
MIME sniffing	X-Content-Type	app.js headers
Archivos maliciosos	Multer tipo+tamaño	config/upload.js

Capítulo 7

Decisiones de Diseño Técnico

7.1. ¿Por qué Node.js?

Tabla 7.1: Comparativa de tecnologías backend consideradas

Criterio	Node.js	Python/Django	Java/Spring	PHP/Laravel
Rendimiento I/O	Excelente	Bueno	Bueno	Medio
Modelo concu- rrente	Async/Event Loop	WSGI sync	Threads	Sync
Curva aprendiza- je	Media	Baja	Alta	Baja
Tiempo de inicio	Muy rápido	Medio	Lento	Rápido
Ecosistema npm	Enorme (2M+)	Grande	Grande	Grande
Mismo lenguaje frontend	Sí (JS)	No	No	No
Despliegue	Simple	Simple	Complejo	Simple
Comunidad acti- va	Muy alta	Alta	Alta	Alta

Justificación: Node.js fue elegido por tres razones principales:

1. **Modelo de I/O no bloqueante:** Una API REST cuya carga de trabajo predominante son operaciones de entrada/salida (consultas a MySQL, llamadas al script Python OCR) se beneficia directamente del event loop de Node.js, que maneja miles de solicitudes concurrentes con un único hilo de ejecución [2].

2. **JavaScript en toda la pila:** El equipo de desarrollo ya tenía experiencia en JavaScript, eliminando la curva de aprendizaje de un lenguaje adicional. El conocimiento de JS/Dart (Flutter) se traslada parcialmente a Node.js.
3. **Velocidad de desarrollo:** La combinación Express.js + npm permite prototipar y desplegar endpoints en minutos, crítico para un proyecto de TT con tiempo limitado.

7.2. ¿Por qué Express.js?

Tabla 7.2: Comparativa de frameworks Node.js

Criterio	Express	Fastify	NestJS	Koa
Rendimiento	Alto	Muy alto	Alto	Alto
Curva aprendizaje	Baja	Media	Alta	Baja
Middleware ecosystem	Enorme	Creciente	Grande	Pequeño
Estructura impuesta	Ninguna	Mínima	Alta	Ninguna
Popularidad / documentación	Máxima	Alta	Alta	Media
Madurez	12+ años	6 años	7 años	10 años

Express fue preferido sobre NestJS por su bajo nivel de abstracción, que permite comprender completamente el flujo de cada solicitud, adecuado para un proyecto académico donde la claridad del código es prioritaria. Fastify ofrece mayor rendimiento, pero su ecosistema de middleware es más reducido.

7.3. ¿Por qué MySQL?

Tabla 7.3: Comparativa de bases de datos consideradas

Criterio	MySQL	PostgreSQL	MongoDB
Modelo de datos	Relacional	Relacional	Documental
Integridad referencial	Sí (InnoDB)	Sí	No nativa

Transacciones	Sí	Sí	Parcial
ACID			
JSON nativo	Sí (v8.0)	Sí (jsonb)	Sí (nativo)
Triggers	Sí	Sí	No
Curva aprendizaje	Baja	Media	Baja
Herramientas GUI	Workbench	pgAdmin	Compass
Prevalencia en currículo IPN	Alta	Media	Media

Justificación: Los datos financieros de FINARA presentan relaciones estructuradas y bien definidas (usuario $\rightarrow$ categorías $\rightarrow$ presupuestos $\rightarrow$ gastos), para las cuales el modelo relacional con integridad referencial garantiza consistencia imposible de lograr de forma nativa con MongoDB. Los triggers automáticos implementados (actualización de presupuesto al insertar un gasto) son una funcionalidad clave que solo está disponible en bases de datos relacionales [4].

7.4. ¿Por qué bcrypt?

Tabla 7.4: Comparativa de algoritmos de hash de contraseñas

Criterio	bcrypt	argon2	PBKDF2
Resistencia GPU	Alta	Muy alta	Media
Memory-hard	No	Sí	No
Madurez	25+ años	7 años	20+ años
Soporte npm	Excelente	Bueno	Nativo (crypto)
Configuración	Simple	Compleja	Simple

bcrypt fue elegido sobre argon2 por su madurez, amplia adopción en producción y el paquete npm `bcrypt` con bindings nativos bien mantenidos. Argon2 es teóricamente superior (memory-hard), pero su configuración correcta requiere mayor experiencia [7].

7.5. ¿Por qué Arquitectura por Capas y no Clean Architecture?

El backend sigue una arquitectura por capas (Routes $\rightarrow$ Controllers $\rightarrow$ Database) en lugar de Clean Architecture (Entities $\rightarrow$ Use Cases $\rightarrow$ Adapters $\rightarrow$ Framework) [16] por las siguientes razones:

- **Tamaño del equipo:** Con 2 desarrolladores, la sobrecarga de abstracciones de Clean Architecture (repositorios, casos de uso, interfaces) superaría su beneficio.
- **Plazo académico:** El tiempo disponible en TT1 favorece velocidad de implementación sobre elegancia arquitectónica.
- **Escala esperada:** FINARA no prevé ser ejecutada en múltiples instancias ni con lógica de dominio compleja que justifique el desacoplamiento extremo.

7.6. Decisión de OCR: Python sobre Node.js

El procesamiento OCR se implementó en Python en lugar de buscar una librería equivalente en Node.js por las siguientes razones:

Tabla 7.5: Justificación de Python para el módulo OCR

Aspecto	Python + OpenCV	Node.js (alternativas)
Ecosistema de visión computacional	Maduro: OpenCV, numpy, scikit-image	Limitado: node-canvas, sharp
Wrapper Tesseract	pytesseract (oficial)	node-tesseract-ocr (no oficial)
Mantenimiento	OpenCV: 20+ años	Alternativas: 2–4 años
Documentación	Extensa y académica	Escasa, en inglés solo
Rendimiento	C++ bajo Python	JavaScript puro

La comunicación entre Node.js y Python se implementó mediante `child_process.spawn`, con el script Python leyendo la imagen desde disco y retornando JSON puro por `stdout`. Los logs de debug se envían por `stderr` para no contaminar el canal de datos.

Capítulo 8

Integración con Frontend

8.1. Contexto de Integración

El backend FINARA es consumido actualmente por la **app móvil Flutter** desarrollada en paralelo por el equipo TT 2026-A062. La plataforma web React está planificada para una fase futura. Ambos clientes consumen la misma API REST mediante solicitudes HTTP estándar con JSON.

8.2. Configuración CORS

El middleware CORS del backend permite solicitudes desde los orígenes configurados mediante la variable de entorno `ALLOWED_ORIGINS`. Durante desarrollo, se acepta cualquier origen (*) para facilitar las pruebas locales.

Listing 8.1: Configuración CORS en app.js

```
1 app.use(cors({
2   origin: process.env.ALLOWED_ORIGINS
3     ? process.env.ALLOWED_ORIGINS.split(',')
4     : '*',
5   methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
6   allowedHeaders: ['Content-Type', 'Authorization'],
7 }));
```

Listing 8.2: Variable de entorno para producción

```
1 # .env (produccion)
2 ALLOWED_ORIGINS=https://finara.app,app://finara
```

8.3. Consumo desde la App Móvil Flutter

La app Flutter implementa un servicio `ApiService` que encapsula todas las llamadas al backend. Los ejemplos siguientes muestran los patrones de integración más representativos.

8.3.1. Autenticación y Almacenamiento del Token

Listing 8.3: Login y almacenamiento del JWT en Flutter (Dart)

```
1 // En Flutter (Dart) - auth_service.dart
2 Future<LoginResponse> login(String email, String password) async {
3   final response = await http.post(
4     Uri.parse('http://192.168.1.x:3000/api/auth/login'),
5     headers: {'Content-Type': 'application/json'},
6     body: jsonEncode({'email': email, 'password': password}),
7   );
8
9   if (response.statusCode == 200) {
10    final data = jsonDecode(response.body);
11    final token = data['data']['token'];
12
13    // Almacenar token de forma segura
14    await FlutterSecureStorage().write(key: 'jwt_token', value:
15      token);
16
17    return LoginResponse.fromJson(data['data']);
18  }
19  throw ApiException(jsonDecode(response.body)['error']);
20 }
```

8.3.2. Request Autenticado Estándar

Listing 8.4: Patrón de request autenticado en Flutter

```
1 // Obtener gastos del mes actual
2 Future<List<Expense>> getExpenses({int? month, int? year}) async {
3   final token = await FlutterSecureStorage().read(key:
4     'jwt_token');
5   final now = DateTime.now();
6
7   final uri = Uri.parse(
8     'http://192.168.1.x:3000/api/expenses'
9     '?month=${month ?? now.month}&year=${year ?? now.year}'
10  );
11 }
```

```
9    );
10
11    final response = await http.get(uri, headers: {
12      'Authorization': 'Bearer $token',
13      'Content-Type': 'application/json',
14    });
15
16    if (response.statusCode == 200) {
17      final data = jsonDecode(response.body)['data'] as List;
18      return data.map((e) => Expense.fromJson(e)).toList();
19    }
20    throw ApiException(jsonDecode(response.body)['error']);
21  }
```

8.3.3. Subida de Imagen para OCR

Listing 8.5: Envío de imagen de ticket para escaneo OCR

```
1  // ticket_service.dart
2  Future<TicketScanResult> scanTicket(File imageFile) async {
3    final token = await FlutterSecureStorage().read(key:
4      'jwt_token');
5    final request = http.MultipartRequest(
6      'POST',
7      Uri.parse('http://192.168.1.x:3000/api/tickets/scan'),
8    );
9    request.headers['Authorization'] = 'Bearer $token';
10    request.files.add(
11      await http.MultipartFile.fromPath('images', imageFile.path)
12    );
13
14    final streamedResponse = await request.send();
15    final response = await
16      http.Response.fromStream(streamedResponse);
17
18    if (response.statusCode == 201) {
19      return TicketScanResult.fromJson(
20        jsonDecode(response.body)['data']
21      );
22    }
23    throw ApiException(jsonDecode(response.body)['error']);
24  }
```

8.4. Manejo de Errores desde el Cliente

La Tabla 8.1 describe cómo la app Flutter interpreta cada código de respuesta del backend.

Tabla 8.1: Códigos HTTP de la API y su interpretación en Flutter

Código	Significado	Acción recomendada en Flutter
200	Éxito	Procesar <code>data</code> del JSON
201	Recurso creado	Actualizar lista, navegar a detalle
400	Datos inválidos	Mostrar <code>error</code> del JSON en el formulario
401	No autenticado / Token expirado	Borrar token, redirigir a Login
404	Recurso no encontrado	Mostrar mensaje “No encontrado”
429	Demasiadas solicitudes	Esperar y reintentar (leer <code>Retry-After</code>)
500	Error del servidor	Mostrar mensaje genérico de error

8.5. Formato de Fechas

El backend retorna fechas en formato ISO 8601 (`YYYY-MM-DDTHH:mm:ss.sssZ`) y acepta fechas en formato `YYYY-MM-DD` en los request bodies. La app Flutter utiliza el paquete `intl` para formatear estas fechas según la localidad del usuario (`es_MX`).

8.6. URL Base de la API

Tabla 8.2: URL base según entorno

Entorno	URL Base
Desarrollo local	<code>http://10.0.2.2:3000/api</code> (emulador Android → localhost)
Dispositivo físico	<code>http://192.168.x.x:3000/api</code> (IP local de la PC en la red WiFi)
Producción	<code>https://api.finara.app/api</code> (dominio de producción)

Nota Técnica

En el emulador Android, `localhost` del dispositivo es diferente al `localhost` de la PC anfitriona. Para acceder al backend corriendo en la PC, el emulador usa la IP especial `10.0.2.2`.

Capítulo 9

Historial de Versiones

9.1. Versión 4.0.0 (Marzo 2026)

Optimizaciones y Testing

- **Seguridad:** Implementación de middleware de rate limiting (`rateLimiter.js`) con límites diferenciados por contexto: 10 req/15min en auth, 20 req/15min en OCR, 200 req/15min en API general.
- **Seguridad:** Headers HTTP de seguridad sin dependencia de Helmet: `X-Content-Type-Options`, `X-Frame-Options`, `X-XSS-Protection`, `Referrer-Policy`.
- **Seguridad:** Middleware sanitizador de inputs (`sanitizer.js`) aplicado globalmente.
- **Testing:** Suite de tests unitarios con Jest + Supertest: 5 módulos (auth, debts, alerts, helpers, middleware), 40+ casos, base de datos mockeada con `jest.fn()`.
- **OCR:** Script `LecturaTickets_v3.py` con pipeline adaptativo según resolución de imagen (1x/2x/3x), filtro bilateral paramétrico, corrección de iluminación y detección de ángulo con `HoughLinesP`.
- **BD:** `enableKeepAlive` en el pool MySQL para reconexión automática.
- **Logging:** Middleware de log coloreado por status code con tiempo de respuesta.
- **Documentación:** README completo, `endpoints.json` con los 46 endpoints, esta documentación LaTeX.

Corregido

- OCR: patrón de total ahora tolera errores de Tesseract (J/T, comas decimales, símbolo §).
- OCR: extracción de JSON de stdout ahora ignora caracteres de ruido antes del { inicial.
- CORS: configuración por variable de entorno en lugar de * fijo.

9.2. Versión 3.0.0 (Febrero 2026)

Módulos OCR, Deudas, Alertas y Usuarios

- **Tickets (OCR):** Endpoint POST /api/tickets/scan con integración Node.js → Python mediante `child_process.spawn`. Creación atómica de Ticket + Productos_Tickets + Gasto.
- **Deudas:** CRUD completo con cálculo de `porcentaje_pagado`, `monto_pendiente` y `dias_para_pago`. Endpoint PUT /api/debts/:id/pay con avance de fecha y detección de liquidación.
- **Alertas:** Sistema de alertas con CRUD, marcado individual y masivo, y configuración de notificaciones. Función interna `createAlert()` disponible para otros controladores.
- **Usuarios:** Endpoints de perfil (/api/users/me) y configuración inicial de onboarding (/api/users/initial-config) con creación automática de ingreso recurrente.
- **OCR v2:** Mejoras sobre v1: corrección de stdout contaminado, formato JSON compatible con `ocrService.js`, extracción de total con validación cruzada por monto en letras, detección de 8 productos en ticket OXXO.

Corregido

- OCR v1: stdout contaminado con texto de debug hacía fallar `JSON.parse()` en Node.js.
- Categorías: error al eliminar categorías con gastos vinculados (FK violation) ahora retorna 400 descriptivo.

9.3. Versión 2.0.0 (Febrero 2026)

Módulos Financieros Completos

- **Categorías:** CRUD completo. Bloqueo de eliminación para categorías predefinidas y con gastos.
- **Presupuestos:** CRUD con cálculo de porcentaje utilizado y consulta por categoría.
- **Ingresos:** CRUD con filtro mensual y endpoint de total por período.
- **Gastos:** CRUD con filtros (mes, categoría, recientes), reversión de presupuesto al eliminar.
- **Dashboard:** Resumen financiero mensual, gráfica de gastos por categoría, estado de presupuestos.
- **Triggers MySQL:** `actualizar_presupuesto_gasto` y `verificar_presupuesto_excedido`

9.4. Versión 1.0.0 (Enero 2026)

Infraestructura Base

- Servidor Express con estructura MVC por capas.
- Conexión MySQL con pool de conexiones (`mysql2/promise`).
- Sistema de autenticación JWT completo: `register`, `login`, `logout`, `verify`, `change-password`.
- Creación automática de 10 categorías predefinidas al registrar usuario.
- Esquema SQL completo (`database/schema.sql`) con 10 tablas e integridad referencial.
- Variables de entorno con `dotenv`, `.env.example` documentado.
- Middleware de validación centralizado con `express-validator`.
- Colección Postman con scripts de test automáticos.

9.5. Roadmap Futuro

Tabla 9.1: Mejoras planificadas para fases futuras

Versión	Característica Planificada
5.0.0	Integración con plataforma web React (endpoints ya compatibles)
5.0.0	Migración de rate limiting a Redis para soporte multi-instancia
5.1.0	Notificaciones push mediante Firebase Cloud Messaging
5.1.0	Detección automática de fecha en tickets OCR
5.2.0	Exportación de reportes financieros en formato PDF y Excel
5.2.0	Soporte de múltiples monedas con tasas de cambio en tiempo real
6.0.0	Despliegue en contenedores Docker con docker-compose

Referencias Bibliográficas

- [1] OpenJS Foundation, *Node.js Documentation*, <https://nodejs.org/en/docs/>, Consultado: marzo 2026, 2024.
- [2] OpenJS Foundation, *The Node.js Event Loop, Timers, and process.nextTick()*, <https://nodejs.org/en/docs/guides/event-loop-timers-and-nexttick>, Consultado: marzo 2026, 2024.
- [3] StrongLoop, Inc. / OpenJS Foundation, *Express.js 4.x API Reference*, <https://expressjs.com/en/4x/api.html>, Consultado: marzo 2026, 2024.
- [4] Oracle Corporation, *MySQL 8.0 Reference Manual*, <https://dev.mysql.com/doc/refman/8.0/en/>, Consultado: marzo 2026, 2024.
- [5] Oracle Corporation, *MySQL 8.0 Reference Manual: InnoDB Storage Engine*, <https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html>, Consultado: marzo 2026, 2024.
- [6] A. Sidorenko y contributors, *mysql2: Fast MySQL driver for Node.js*, <https://www.npmjs.com/package/mysql2>, npm package v3.9.2. Consultado: marzo 2026, 2024.
- [7] N. Mates y contributors, *bcrypt: A bcrypt library for NodeJS*, <https://www.npmjs.com/package/bcrypt>, npm package v5.1.1. Consultado: marzo 2026, 2024.
- [8] M. Jones, J. Bradley y N. Sakimura, «JSON Web Token (JWT),» Internet Engineering Task Force (IETF), RFC 7519, mayo de 2015.
- [9] S. Mott y contributors, *dotenv: Loads environment variables from .env file*, <https://www.npmjs.com/package/dotenv>, npm package v16.4.5. Consultado: marzo 2026, 2024.
- [10] express-validator contributors, *express-validator: An express.js middleware for validator.js*, <https://express-validator.github.io/docs/>, Consultado: marzo 2026, 2024.
- [11] Meta Platforms, Inc., *Jest: Delightful JavaScript Testing Framework*, <https://jestjs.io/docs/getting-started>, Consultado: marzo 2026, 2024.

- [12] visionmedia and contributors, *SuperTest: HTTP assertions made easy via superagent*, <https://www.npmjs.com/package/supertest>, npm package v6.3.4. Consultado: marzo 2026, 2024.
- [13] OpenCV team, *OpenCV Documentation: Image Processing in OpenCV*, https://docs.opencv.org/4.x/d2/d96/tutorial_py_table_of_contents_imgproc.html, Consultado: marzo 2026, 2024.
- [14] Google LLC and Tesseract contributors, *Tesseract OCR Documentation*, <https://tesseract-ocr.github.io/tessdoc/>, Consultado: marzo 2026, 2024.
- [15] NumPy contributors, *NumPy Documentation*, <https://numpy.org/doc/stable/>, Consultado: marzo 2026, 2024.
- [16] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017, ISBN: 9780134494166.
- [17] OWASP Foundation, *Cross Site Scripting (XSS)*, <https://owasp.org/www-community/attacks/xss/>, Consultado: marzo 2026, 2021.
- [18] OWASP Foundation, *Blocking Brute Force Attacks*, https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks, Consultado: marzo 2026, 2021.
- [19] OWASP Foundation, *OWASP Secure Headers Project*, <https://owasp.org/www-project-secure-headers/>, Consultado: marzo 2026, 2023.
- [20] OWASP Foundation, *SQL Injection*, https://owasp.org/www-community/attacks/SQL_Injection, Consultado: marzo 2026, 2021.